# Case: Forecasting US Payroll Growth with a Neural Network

Forecasting with Machine Learning, Case 1

## Objective

Implement a feedforward neural network for a time-series regression from start to finish, and investigate whether it improves forecasting performance over simple benchmarks. The tools are: The slides, notebook, and exercises of Lecture 4, in particular: temporal splitting, standardization on the training sample only, the `DataLoader`, the training and validation loop, and the regularization methods early stopping, dropout, $\ell_1$, and $\ell_2$. A few hints are provided below the conclusion. You may reuse your own functions from the exercises.

The case is not to be handed in, nor will you receive feedback. You are encouraged to work in groups, and to discuss the problems with your fellow students. You will be provided with suggested solutions to the problems, but we recommend you attempt to solve the problems on your own first. The solutions will be posted around Wednesday next week. Feel free to reach out to Natasia, nvilladsen@econ.au.dk, if you have questions.

## Data

You will use the file `payroll_case.csv`, built from the 2026-09 vintage of FRED-MD (McCracken and Ng, 2016), a monthly database of US macroeconomic series. The file can be read in directly from the course GitHub with `pd.read_csv("https://raw.githubusercontent.com/nvilladsen/fml-2026/main/assets/data/payroll_case.csv")`. The target variable, `y`, is monthly growth of total nonfarm payroll employment (`PAYEMS` – this is the standard measure of employment in the U.S.) in month $t+1$, $100\,(\log \mathrm{PAYEMS}_{t+1} - \log \mathrm{PAYEMS}_t)$. It is assumed that the inputs in row $t$ are known at *the end of* month $t$.

This means that **lagging of the predictors is not necessary**. Variables are transformed to stationarity with the FRED-MD transformation codes (this is done before the data reaches us, so we need not handle this in our code. All this means is that they have solved a set of statistical challenges for us). Publication lags and data revisions are ignored. CPI is missing for October–December 2025 in this vintage, so the last row of the file is September 2025.

The file contains `y`, plus 42 inputs: 12 lags of payroll growth (`own_l0`, …, `own_l11`), and lags 0, 1, and 2 of unemployment, industrial production, CPI inflation, housing starts, the federal funds rate, the 10-year yield, M2, the S&P 500, capacity utilization, and the term spread (`spread_l0`, …).

The sample is to be split as follows: training 1960–1999, validation 2000–2009, test 2010–September 2025.

## Problems

Answer the question at the end of each part in a few sentences. Collect the answers, the key figures and tables, and your code in one (markdown) notebook.

## Problem 1: Reading and splitting the data

a. Read in `payroll_case.csv`, convert the date to a date object and set it as the index. Inspect the data. Which column is the target?
b. Separate the data into target and predictors (`y` and `X`). Split the data chronologically into training, validation and test set, and print their shapes.

## Problem 2: Data

a. Plot `y` and highlight the three datasets on the plot (train/validation/test).
b. Report the mean and standard deviation of `y` in each sample.
c. What do the three samples have in common, and how do they differ?
d. Which features of the test sample could be a problem for a model estimated on 1960 – 1999 data?

## Problem 3: Benchmarks

Note that the lag of the target is included in the predictors as `own_l0`. The training sample mean forecast is used both in 3.a.ii as a forecast and in 3.b as a benchmark for the out-of-sample $R^2$.

a. Construct three benchmark forecasts (of the test sample):
   **i)** the random walk, as in Lecture 3, $\hat{y}_{t+1} = y_t$
   **ii)** the training-sample mean
   **iii)** OLS of $y$ on the predictors, estimated on the training sample

b. Report the test MSE and the $R^2_{\text{OOS}}$ relative to the training-sample mean – as in Lecture 4, Exercise 17 – for the three benchmark models.

## Problem 4: Standardization and DataLoaders

a. Standardize the inputs and the target.
b. Convert the standardized data to PyTorch tensors, and build a `TensorDataset` and a `DataLoader` for each of the three samples.
c. Does the test sample contain standardized inputs far outside the training range? Check this empirically.

## Problem 5: The network

a. Implement and run a feedforward network with three *hidden* layers (select the number of hidden neurons yourself) and ReLU activation function for the first two layers, and a tanh activation function for the third layer, based on the slides, notebook, and exercises of Lecture 4. Use `nn.MSELoss` as the criterion and Adam as the optimizer. You decide whether to fix the number of epochs or to use early stopping.
b. Plot the training and validation losses over the epochs.
c. Report the test MSE after transforming the forecasts back to the original scale.

## Problem 6: Regularization

a. Train the network of Problem 5 under five settings, all from the same seed, and with the same way of terminating the training as in Problem 5 (fixed number of epochs or early stopping). Implement the methods one at a time, resetting the remaining settings for each new setting.
   **i)** no regularization (the network of Problem 5, to have the correct seed)
   **ii)** dropout with $p = 0.5$ after the first hidden layer
   **iii)** $\ell_2$ regularization with `weight_decay=0.01`
   **iv)** $\ell_1$ regularization with $\lambda = 0.0001$
   **v)** all three of ii), iii) and iv) in conjunction

Considering that you will need to implement dropout, and you are to implement both training and evaluation five times, you may want to redefine the network class, as well as define a training function, returning the trained model and the training and validation losses, and an evaluation function, returning the test forecasts (in original scale), the MSE, and the $R^2_{\text{OOS}}$.

b. Report the validation loss of each of the five networks, and select the one with the lowest validation loss. This network should be used in the remainder of the case.

c. Count the parameters of this network (See Exercise 10). The training sample has about 480 observations: why do you expect regularization to matter?

## Problem 7: Seeds and ensembles

a. Train the network selected in Problem 6 with 5 different seeds (select the seeds yourself. To match the suggested solutions, use the seeds {0, 1, 2, 3, 4}).
b. Report the test MSE of each seed, and the test MSE of the average forecast (the ensemble).
c. How large is the variance of the test MSE across seeds? What does that imply for conclusions drawn from a single run?

## Problem 8: Evaluation

a. Add the ensemble of Problem 7 to the table of Problem 3, with the test MSE and the out-of-sample $R^2$ relative to the training-sample mean.
b. Plot $y$ over the test sample together with the forecasts of the network and of the training-sample mean. Does the network follow the movements of $y$, or does it stay close to the mean?
c. Does the network beat the benchmarks? In which months does it gain or lose relative to the mean? Link your answer to your findings in Problems 2 and 4.

# Conclusion

Consider the following questions:

- Does the neural network beat the benchmarks?
- How stable is the network across seeds?
- Under which conditions would you expect a network to be worth it in a forecasting problem like this one?
- What is the main limitation of the design (think, e.g., the fixed estimation window?)

## Hints

- Session 4 slides: “Temporally Conscious Data Splitting”, “Standardization”, “Main Training and Validation Loop”, “Early Stopping II”, “Guards Against Overfitting”.
- Store the training and validation losses of each epoch in lists. You need them to plot the losses (Problem 5.b).
- Set a seed before you create each network, so that the networks of Problem 6 differ only in their regularization, and so that your results can be reproduced.
- Remember to transform the forecasts back to the original scale before calculating the test MSE (Problem 5.c and 6.a).

## Reference

McCracken, M. W. and Ng, S. (2016). FRED-MD: A monthly database for macroeconomic research. *Journal of Business and Economic Statistics*, 34(4), 574–589.